Denoising Diffusion Probabilistic Models (DDPMs)

Notes for ChemDM

1 Setup

We have data $\boldsymbol{x}_0 \sim q(\boldsymbol{x}_0)$ in $\mathbb{R}^d$ and want to learn a generative model that can produce new samples from $q(\boldsymbol{x}_0)$. The idea is to *gradually destroy* the data by adding noise (the forward process) and then *learn to reverse* that destruction (the reverse process).

2 Forward process

Define a sequence of noise levels through a **variance schedule** $\beta_1, \beta_2, \ldots, \beta_T \in (0, 1)$. The forward process adds Gaussian noise in T steps:

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}\Big(\boldsymbol{x}_t;\ \sqrt{1-\beta_t}\,\boldsymbol{x}_{t-1},\ \beta_t \boldsymbol{I}\Big)\,. \tag{1}$$

Each step shrinks the signal by $\sqrt{1-\beta_t}$ and adds noise with variance β_t. After many steps, $\boldsymbol{x}_T$ is approximately standard Gaussian regardless of $\boldsymbol{x}_0$.

2.1 Closed-form sampling at arbitrary t

Define $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. Then we can jump directly from $\boldsymbol{x}_0$ to $\boldsymbol{x}_t$ without simulating intermediate steps:

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = \mathcal{N}\big(\boldsymbol{x}_t;\ \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0,\ (1-\bar{\alpha}_t)\boldsymbol{I}\big)\,, \tag{2}$$

or equivalently,

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}). \tag{3}$$

This is the **reparameterization trick** and is what we use during training to generate noisy samples.

Proof sketch. Each forward step is a linear Gaussian map. Composing t of them gives $\boldsymbol{x}_t = \big(\prod_{s=1}^{t} \sqrt{\alpha_s}\big)\boldsymbol{x}_0 + \text{noise}$, with the noise terms accumulating into a single Gaussian with variance $1 - \prod_{s=1}^{t} \alpha_s = 1 - \bar{\alpha}_t$.

2.2 Variance schedule

Two common choices:

- **Linear:** β_t increases linearly from $\beta_1 \approx 10^{-4}$ to $\beta_T \approx 0.02$.
- **Cosine** (Nichol & Dhariwal, 2021): define $\bar{\alpha}_t$ directly as

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \qquad f(t) = \cos^2\!\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right),$$

with a small offset $s = 0.008$ to avoid β_t becoming too small near $t = 0$. Then $\beta_t = 1 - \bar{\alpha}_t / \bar{\alpha}_{t-1}$. This schedule spends more of the diffusion budget at intermediate noise levels, where the denoising task is most informative.

3 Reverse process

The key insight: if β_t is small, each forward step $q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1})$ is nearly Gaussian, so the true reverse step $q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ is *also* nearly Gaussian. We can therefore parameterize a learned reverse:

$$p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t) = \mathcal{N}\big(\boldsymbol{x}_{t-1};\ \boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t),\ \sigma_t^2 \boldsymbol{I}\big)\,, \tag{4}$$

where $\boldsymbol{\mu}_\theta$ is a neural network and σ_t^2 is set to a known value (see below). Sampling starts from $\boldsymbol{x}_T \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ and iterates $\boldsymbol{x}_{t-1} \sim p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ for $t = T, T-1, \ldots, 1$.

3.1 The true posterior

If we *knew* $\boldsymbol{x}_0$, the reverse step would be tractable:

$$q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}\Big(\boldsymbol{x}_{t-1};\ \tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0),\ \tilde{\beta}_t \boldsymbol{I}\Big)\,, \tag{5}$$

with

$$\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1 - \bar{\alpha}_t}\,\boldsymbol{x}_0 + \frac{\sqrt{\alpha_t}\,(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\,\boldsymbol{x}_t, \tag{6}$$

$$\tilde{\beta}_t = \frac{(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\,\beta_t. \tag{7}$$

The posterior mean is a weighted average of $\boldsymbol{x}_0$ (the clean data) and $\boldsymbol{x}_t$ (the current noisy state).

4 Training objective

4.1 Variational bound

The DDPM training objective is derived from the variational lower bound (ELBO) on $\log p_\theta(\boldsymbol{x}_0)$. After standard manipulations, it decomposes into T KL-divergence terms, each comparing the learned reverse $p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ against the true posterior $q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0)$. Since both are Gaussians with the same (fixed) variance $\tilde{\beta}_t$, the KL reduces to a squared difference between their means.

The upshot: training reduces to teaching $\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t)$ to approximate $\tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t, \boldsymbol{x}_0)$.

4.2 Parameterization choices

We can parameterize $\boldsymbol{\mu}_\theta$ in several equivalent ways. From (3) we can express $\boldsymbol{x}_0$ and $\boldsymbol{\varepsilon}$ in terms of each other:

$$\boldsymbol{x}_0 = \frac{\boldsymbol{x}_t - \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\varepsilon}}{\sqrt{\bar{\alpha}_t}}, \qquad \boldsymbol{\varepsilon} = \frac{\boldsymbol{x}_t - \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0}{\sqrt{1 - \bar{\alpha}_t}}. \tag{8}$$

$\boldsymbol{\varepsilon}$-prediction (Ho et al., 2020). Let the network predict the noise: $\boldsymbol{\varepsilon}_\theta(\boldsymbol{x}_t, t) \approx \boldsymbol{\varepsilon}$. Substituting into (6):

$$\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\varepsilon}_\theta(\boldsymbol{x}_t, t) \right). \tag{9}$$

The simplified training loss is:

$$L_{\text{simple}} = \mathbb{E}_{t,\, \boldsymbol{x}_0,\, \boldsymbol{\varepsilon}} \left[\left\| \boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(\boldsymbol{x}_t, t) \right\|^2 \right], \tag{10}$$

where $t \sim \text{Uniform}\{1, \ldots, T\}$, $\boldsymbol{x}_0 \sim q(\boldsymbol{x}_0)$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, and $\boldsymbol{x}_t$ is given by (3).

$\boldsymbol{x}_0$-prediction. Let the network predict the clean data directly: $\hat{\boldsymbol{x}}_0 = f_\theta(\boldsymbol{x}_t, t) \approx \boldsymbol{x}_0$. The training loss is:

$$L_{x_0} = \mathbb{E}_{t,\, \boldsymbol{x}_0,\, \boldsymbol{\varepsilon}} \left[\left\| \boldsymbol{x}_0 - f_\theta(\boldsymbol{x}_t, t) \right\|^2 \right]. \tag{11}$$

At sampling time, plug the predicted $\hat{\boldsymbol{x}}_0$ into the posterior mean (6) to get $\boldsymbol{\mu}_\theta$, then sample:

$$\boldsymbol{x}_{t-1} = \tilde{\boldsymbol{\mu}}_t(\boldsymbol{x}_t,\, \hat{\boldsymbol{x}}_0) + \sqrt{\tilde{\beta}_t}\, \boldsymbol{z}, \qquad \boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{12}$$

with no noise added at $t = 1$ (i.e. we return $\hat{\boldsymbol{x}}_0$ directly).

Equivalence. The two parameterizations are related by the linear map (3). The losses (10) and (11) differ only by a t-dependent reweighting factor $\bar{\alpha}_t/(1-\bar{\alpha}_t)$. In practice, $\boldsymbol{x}_0$-prediction tends to work better when combined with a cosine schedule, while $\boldsymbol{\varepsilon}$-prediction is the original DDPM formulation.

5 Training algorithm

The full training procedure is:

1. Sample a data point $\boldsymbol{x}_0 \sim q(\boldsymbol{x}_0)$.
2. Sample a random timestep $t \sim \text{Uniform}\{0, 1, \ldots, T-1\}$.
3. Sample noise $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.
4. Compute $\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\, \boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\, \boldsymbol{\varepsilon}$.
5. Predict $\hat{\boldsymbol{x}}_0 = f_\theta(\boldsymbol{x}_t, t) \quad$ (or $\hat{\boldsymbol{\varepsilon}} = \boldsymbol{\varepsilon}_\theta(\boldsymbol{x}_t, t)$).
6. Compute loss: $\|\boldsymbol{x}_0 - \hat{\boldsymbol{x}}_0\|^2 \quad$ (or $\|\boldsymbol{\varepsilon} - \hat{\boldsymbol{\varepsilon}}\|^2$).
7. Backpropagate and update θ.

Note that each gradient step uses a *single random t*. Over many iterations the network learns to denoise at all noise levels.

6 Sampling algorithm

1. Sample $\boldsymbol{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.
2. For $t = T, T-1, \ldots, 1$:
   (a) Predict $\hat{\boldsymbol{x}}_0 = f_\theta(\boldsymbol{x}_t, t)$.
   (b) Compute the posterior mean $\tilde{\boldsymbol{\mu}}_t$ using (6) with $\hat{\boldsymbol{x}}_0$.
   (c) Sample $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ if $t > 1$, else $\boldsymbol{z} = \mathbf{0}$.
   (d) Set $\boldsymbol{x}_{t-1} = \tilde{\boldsymbol{\mu}}_t + \sqrt{\tilde{\beta}_t}\, \boldsymbol{z}$.
3. Return $\boldsymbol{x}_0$.

7 Connection to score-based models

The **score** of a distribution is $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$. For the forward marginal $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$ from (2):

$$\nabla_{\boldsymbol{x}_t} \log q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = -\frac{\boldsymbol{x}_t - \sqrt{\bar{\alpha}_t}\, \boldsymbol{x}_0}{1 - \bar{\alpha}_t} = -\frac{\boldsymbol{\varepsilon}}{\sqrt{1 - \bar{\alpha}_t}}. \tag{13}$$

So predicting $\boldsymbol{\varepsilon}$ is equivalent to estimating the score (up to a known scaling). Score-based generative models (Song & Ermon, 2019) take this perspective and work in continuous time, replacing the discrete chain with a stochastic differential equation (SDE). The DDPM discrete chain can be seen as an Euler–Maruyama discretization of that SDE.

8 Application to transition paths

In our setting, $\boldsymbol{x}_0 = \boldsymbol{x}(s) \in \mathbb{R}^{N_{\text{atoms}} \times 3}$ is the atomic configuration at normalized arclength s along a transition path. The denoising network f_θ is conditioned on:

- $\boldsymbol{x}_A$, $\boldsymbol{x}_B$: reactant and product geometries (fixed graphs),
- $s \in [0, 1]$: position along the path,
- t: diffusion noise level.

The network predicts the clean configuration $\hat{\boldsymbol{x}}_0 = f_\theta(\boldsymbol{x}_t, \boldsymbol{x}_A, \boldsymbol{x}_B, s, t)$.

Because the reverse process is *stochastic*, sampling with the same $(\boldsymbol{x}_A, \boldsymbol{x}_B, s)$ but different random seeds produces **different transition paths**. This is precisely the property we want: for a given reaction, multiple mechanistically valid pathways can exist, and a diffusion model can represent this multi-modality naturally, whereas a deterministic regression model is forced to average over them.

At the boundaries, the conditioning enforces $\hat{\boldsymbol{x}}_0 \approx \boldsymbol{x}_A$ when $s \approx 0$ and $\hat{\boldsymbol{x}}_0 \approx \boldsymbol{x}_B$ when $s \approx 1$, regardless of the noise level t.

Key references

- Ho, Jain & Abbeel. *Denoising Diffusion Probabilistic Models.* NeurIPS, 2020.
- Nichol & Dhariwal. *Improved Denoising Diffusion Probabilistic Models.* ICML, 2021.
- Song & Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution.* NeurIPS, 2019.
- Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations.* ICLR, 2021.